

1/9/26 DAY 9:

Copy Assignment Operator

- When an object is assigned to another object of the same type, the copy assignment operator is called.
- The copy assignment operator is a special member function that is automatically generated by the compiler if not explicitly defined by the programmer.
- The copy assignment operator performs a member-wise copy of the data members from the source object to the destination object.

Syntax:

```
ClassName& operator=(const ClassName& other);
```

Assignment Operator =

- The assignment operator is used to assign the value of one object to another object of the same type.
- It is a binary operator that takes two operands: the left-hand side (LHS) and the right-hand side (RHS).

```
ClassName obj1, obj2;  
obj1 = obj2; // Calls the copy assignment operator
```

Resource Ownership and Copying

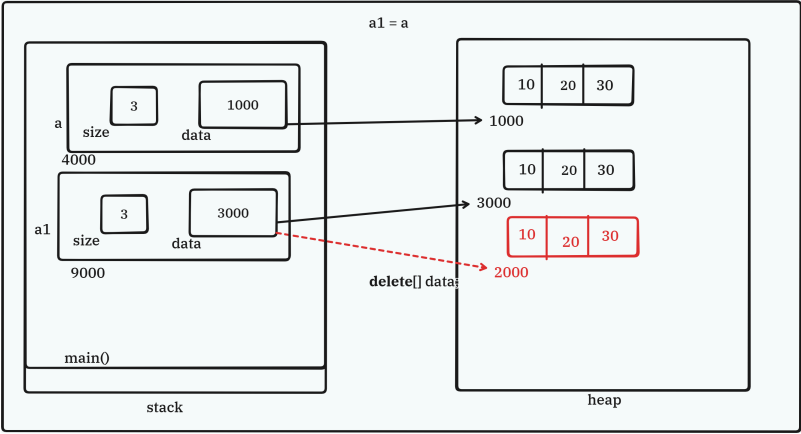
- When an object owns resources (like dynamic memory, file handles, etc.), the default copy assignment operator may lead to issues like double deletion or resource leaks.
- To handle resource ownership correctly, it is often necessary to define a custom copy assignment operator that properly manages the resources during assignment.

Rule of Three — Introduction

The Rule of Three states that if a class defines one of the following special member functions, it should probably explicitly define all three:

1. Destructor
2. Copy Constructor
3. Copy Assignment Operator

When a class manages resources, it is important to ensure that these special member functions are properly defined to avoid resource management issues.



Lvalue

- An lvalue (locator value) refers to an object that occupies some identifiable location in memory (i.e., it has an address).
- Lvalues can appear on the left-hand side of an assignment expression, meaning they can be assigned a value.

Rvalue

- An rvalue (right-value) refers to an object that does not have a persistent location in memory (i.e., it does not have an address).
- Rvalues are typically temporary objects or literals that cannot be assigned a value.

Lvalue vs Rvalue

```
int x = 5; // x is an lvalue
int y = x; // y is an lvalue
int z = 5; // z is an rvalue

int a = x; // x is an lvalue

int& b = a; // a is an lvalue
int& c = 5; // Error: 5 is an rvalue and cannot bind to an lvalue reference

int && d = 5; // d is an rvalue reference, and 5 is an rvalue
```

Lvalue	Rvalue
Has a memory address	Does not have a memory address
Can appear on the left-hand side of an assignment	Cannot appear on the left-hand side of an assignment
Can be assigned a value	Cannot be assigned a value
Can be used to initialize references	Cannot be used to initialize references
Can be used to access the value of an object	Cannot be used to access the value of an object

Lvalue	Rvalue
Can be used to modify the value of an object	Cannot be used to modify the value of an object

Rvalue References

- Rvalue references are a type of reference that can bind to rvalues (temporary objects) and allow for efficient resource management and move semantics.
- Rvalue references are declared using the `&&` syntax.

```
int&& rvalue_ref = 5; // 5 is an rvalue, rvalue_ref is an rvalue reference
```

std::move

- **std::move** is a utility function in the C++ Standard Library that casts an object to an rvalue reference, enabling move semantics.
- It is used to indicate that an object can be "moved from," allowing the resources of the object to be transferred rather than copied.

```
int x = 5;  
int&& y = std::move(x); // x is moved to y
```

Move Semantics

- Move semantics is a programming technique that allows the resources of an object to be transferred (moved) rather than copied, improving performance and efficiency.
- Move semantics is particularly useful for classes that manage dynamic resources, such as memory or file handles.
- Move semantics is implemented using move constructors and move assignment operators, which are special member functions that handle the transfer of resources from one object to another.

Move Constructor

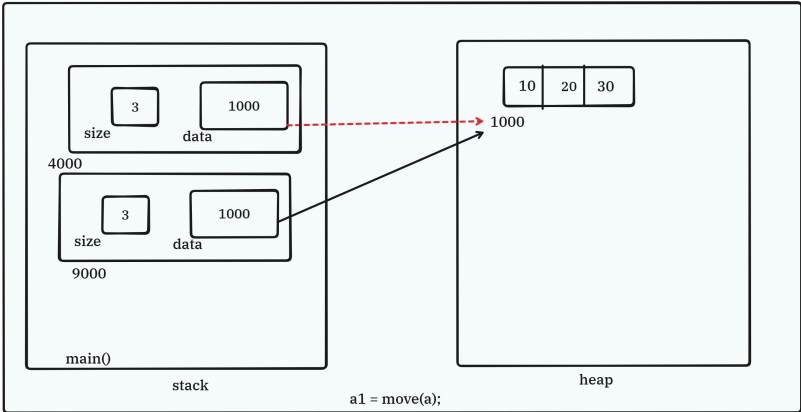
- A move constructor is a special member function that initializes a new object by transferring the resources of an existing object (the source) to the new object (the destination).
- The move constructor is called when an object is initialized with an rvalue (temporary object)

```
ClassName(className&& other); // Move constructor
```

Move Assignment

- A move assignment operator is a special member function that assigns the resources of an existing object (the source) to another object (the destination).
- The move assignment operator is called when an object is assigned an rvalue (temporary object).

```
ClassName& operator=(ClassName&& other); // Move assignment operator
```



Copy vs Move

Copy	Move
Creates a new copy of the object	Transfers ownership of resources from one object to another
Can be expensive for large objects	Can be more efficient for large objects
May lead to resource duplication	Avoids resource duplication
May lead to resource leaks if not handled properly	Helps manage resources more efficiently
Can be used with lvalues	Can be used with rvalues
Can be used with const objects	Cannot be used with const objects

Rule of Five — Introduction

The Rule of Five states that if a class defines one of the following special member functions, it should probably explicitly define all five:

- 1. Destructor
- 2. Copy Constructor
- 3. Copy Assignment Operator
- 4. Move Constructor
- 5. Move Assignment Operator

Static Data Members

- Static data members are class-level variables that are shared among all instances of a class.
- They are declared using the **static** keyword and have a single copy that is shared by all objects of the class.
- Static data members are typically used to store information that is common to all objects of a class, such as a count of the number of objects created or a shared configuration setting.

- Static data members are initialized outside the class definition, typically in a source file.
- Static data members can be accessed using the class name followed by the scope resolution operator `::`.
- Static data members can also be accessed using an object of the class, but it is not recommended, as it can lead to confusion about the variable's behavior.

```
class MyClass {
public:
    static int staticVar; // Declaration
};

// to define the static member variable, you need to do it outside the class
// definition, typically in a source file (.cpp)
int MyClass::staticVar = 0; // Definition
```

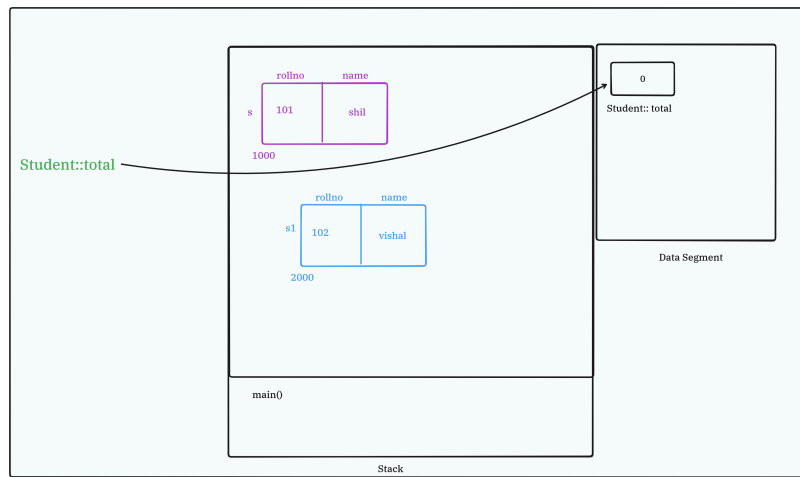
Static Member Functions

- Static member functions are class-level functions that can be called without creating an instance of the class.
- They are declared using the **static** keyword and can only access static data members and other static member functions of the class.
- Static member functions cannot access non-static data members or member functions of the class, as they do not have access to the **this** pointer.
- Static member functions are typically used to perform operations that are related to the class as a whole, rather than to individual objects of the class.
- Static member functions can be called using the class name followed by the scope resolution operator `::`.
- Static member functions can also be called using an object of the class, but it is not recommended, as it can lead to confusion about the function's behavior.

Static Member Functions can access only static data members and other static member functions of the class. They cannot access non-static data members or member functions of the class, as they do not have access to the **this** pointer.

```
class MyClass {
public:
    static void staticFunc(); // Declaration
};

// to define the static member function, you need to do it outside the class
// definition, typically in a source file (.cpp)
// can be in class definition as well
void MyClass::staticFunc() {
    // Can access static data members and other static member functions
}
```



Class-level vs Object-level Data

Class-level Data	Object-level Data
Shared among all instances of a class	Unique to each instance of a class
Declared using the static keyword	Declared without the static keyword
Initialized outside the class definition	Initialized in the constructor or member initializer list
Can be accessed using the class name followed by the scope resolution operator ::	Can be accessed using an object of the class
Can be accessed using an object of the class, but it is not recommended	Can only be accessed using an object of the class
Can be used to store information that is common to all objects of a class	Can be used to store information that is unique to each object of a class

Constant Data Members

- Constant data members are member variables of a class that cannot be modified after they are initialized.
- They are declared using the **const** keyword and must be initialized in the constructor's member initializer list or at the point of declaration (if they are static).
- Constant data members are typically used to represent values that should not change throughout the lifetime of an object, such as configuration settings or fixed parameters.

Syntax:

```
class MyClass {  
public:  
    const int constantVar; // Declaration  
  
    MyClass(int value) : constantVar(value) {} // Initialization in constructor's  
    member initializer list  
};
```

Constant Member Functions

- Constant member functions are member functions of a class that do not modify the state of the object.
- They are declared using the **const** keyword after the function's parameter list.
- Constant member functions can be called on both const and non-const objects, but they cannot modify any non-static data members of the class or call any non-const member functions.

Syntax:

```
class MyClass {  
public:  
    void nonConstFunc(); // Non-const member function  
    void constFunc() const; // Const member function  
};
```

Const Objects

- Const objects are instances of a class that cannot be modified after they are created.
- They are declared using the **const** keyword and can only call constant member functions of the class.
- Const objects are useful for ensuring that certain data remains unchanged throughout the program's execution.

```
const MyClass obj; // Const object
```

mutable Members

- Mutable members are member variables of a class that can be modified even in constant member functions or through const objects.
- They are declared using the **mutable** keyword and are typically used for caching or lazy evaluation, where the value of the member may change even though the object is considered constant.

Syntax:

```
class MyClass {  
public:  
    mutable int mutableVar; // Declaration
```

```
void constFunc() const {
    mutableVar = 10; // Allowed, as mutableVar is declared as mutable
}
};
```

Operator Overloading

- Operator overloading is a feature in C++ that allows developers to redefine the behavior of operators for user-defined types (classes and structs).
- By overloading operators, you can provide intuitive and meaningful operations for your custom types, making them easier to use and understand.

Single operands	Unary =	++, --, -, !										
Two operands	Binary =	<table border="0"> <tr> <td>Arithmetic</td> <td>+, -, *, /, %</td> </tr> <tr> <td>Logical</td> <td>&&, </td> </tr> <tr> <td>Bitwise</td> <td>&, , XOR, NAND</td> </tr> <tr> <td>Relational</td> <td><, >, <=, >=, ==, !=</td> </tr> <tr> <td>Assignment</td> <td>+=, -=, *=, /=, %=</td> </tr> </table>	Arithmetic	+, -, *, /, %	Logical	&&,	Bitwise	&, , XOR, NAND	Relational	<, >, <=, >=, ==, !=	Assignment	+=, -=, *=, /=, %=
Arithmetic	+, -, *, /, %											
Logical	&&,											
Bitwise	&, , XOR, NAND											
Relational	<, >, <=, >=, ==, !=											
Assignment	+=, -=, *=, /=, %=											
	Ternary =	cond ? statement : statement										
	Special =	.-> :: .* ->* () [] sizeof typedef () [] >> <<										

Why Operator Overloading?

- Operator overloading allows you to define how operators behave for your custom types, enabling you to use familiar syntax and semantics when working with objects of those types.
- It can improve code readability and maintainability by allowing you to express operations in a more natural way, similar to how built-in types are used.

Unary Operators

- Unary operators are operators that operate on a single operand. In C++, you can overload unary operators for user-defined types to provide custom behavior.

Common unary operators that can be overloaded include:

- + (unary plus)
- - (unary minus)
- ! (logical NOT)
- ~ (bitwise NOT)
- ++ (pre-increment and post-increment)
- -- (pre-decrement and post-decrement)

Binary Operators

- Binary operators are operators that operate on two operands. In C++, you can overload binary operators for user-defined types to provide custom behavior.

Common binary operators that can be overloaded include:

Arithmetic Operators

- `+` (addition)
- `-` (subtraction)
- `*` (multiplication)
- `/` (division)
- `%` (modulus)

Relational Operators

- `==` (equality)
- `!=` (inequality)
- `<` (less than)
- `>` (greater than)
- `<=` (less than or equal to)
- `>=` (greater than or equal to)

Logical Operators

- `&&` (logical AND)
- `||` (logical OR)
- `!` (logical NOT)

Assignment Operators

- `=` (assignment)
- `+=` (addition assignment)
- `-=` (subtraction assignment)
- `*=` (multiplication assignment)
- `/=` (division assignment)
- `%=` (modulus assignment)

Operators that cannot be overloaded:

- `::` (scope resolution)
- `.` (member access)
- `.*` (member pointer access)
- `?:` (ternary conditional)
- `sizeof` (size of)
- `typeid` (type identification)
- `const_cast`, `dynamic_cast`, `reinterpret_cast`, `static_cast` (type casting operators)

Arithmetic Operator Overloading

- Arithmetic operator overloading allows you to define custom behavior for arithmetic operations involving user-defined types. This can make your classes more intuitive and easier to use.

```
class Complex {
    int real;
    int imag;
public:
    Complex(int r = 0, int i = 0) : real(r), imag(i) {}

    // Overload the + operator
    Complex operator+(const Complex& other) {
        return Complex(real + other.real, imag + other.imag);
    }
};
```

Relational Operator Overloading

- Relational operator overloading allows you to define custom behavior for comparison operations involving user-defined types. This can make your classes more intuitive and easier to use.

```
class Complex {
    int real;
    int imag;
public:
    Complex(int r = 0, int i = 0) : real(r), imag(i) {}
    // Overload the == operator
    bool operator==(const Complex& other) {
        return (real == other.real) && (imag == other.imag);
    }
};
```

Assignment Operator Overloading

- Assignment operator overloading allows you to define custom behavior for assignment operations involving user-defined types. This can make your classes more intuitive and easier to use.

```
class Complex {
    int real;
    int imag;
public:
    Complex(int r = 0, int i = 0) : real(r), imag(i) {}

    // Overload the assignment operator
    Complex& operator=(const Complex& other) {
        if (this != &other) { // Self-assignment check
            real = other.real;
            imag = other.imag;
        }
        return *this;
    }
};
```

```
}  
};
```

Pre-increment / Post-increment

- Pre-increment and Post-increment operator overloading allows you to define custom behavior for increment operations involving user-defined types. This can make your classes more intuitive and easier to use.

```
class Counter {  
    int count;  
public:  
    Counter(int c = 0) : count(c) {}  
  
    // Overload the pre-increment operator  
    Counter& operator++() {  
        ++count;  
        return *this;  
    }  
  
    // Overload the post-increment operator  
    Counter operator++(int) {  
        Counter temp(*this);  
        ++count;  
        return temp;  
    }  
};
```

Tomorrow's Topics:

- Member vs Non-member Operators
- Subscript Operator []
- Stream Operators << and >>
- Friend Functions
- Friend Classes
- Abstraction Concept
- Association
- Aggregation
- Composition
- Relationship Modeling
- Inheritance Concept
- Base Class and Derived Class
- Modes of Inheritance
- Types of Inheritance
- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance

- Hierarchical Inheritance
- Hybrid Inheritance